



Particle Swarm Optimization

Nature-inspired optimization algorithm

Present By:

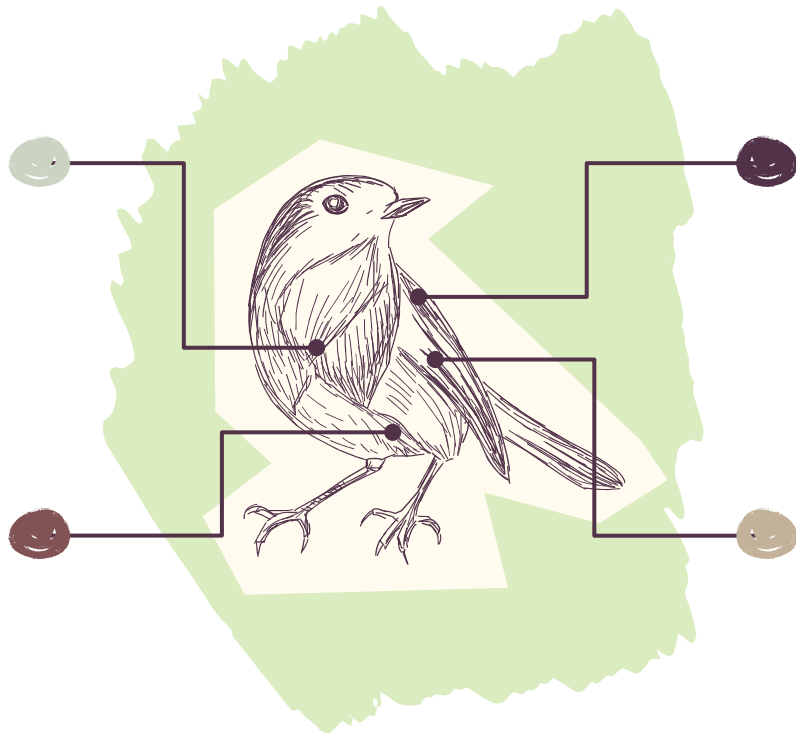
- **Jeany HAN, 1820249024**
- **Inna Karamova, 1820249048**
- **Mathilde HELENE, 1820249015**
- **Fernando Nava, 1820249040**

Collective animal behavior

Moves smoothly
Never collide

It's not about the leader

Swarm intelligence
Quite simple rules
Follow its own kind
Inform others

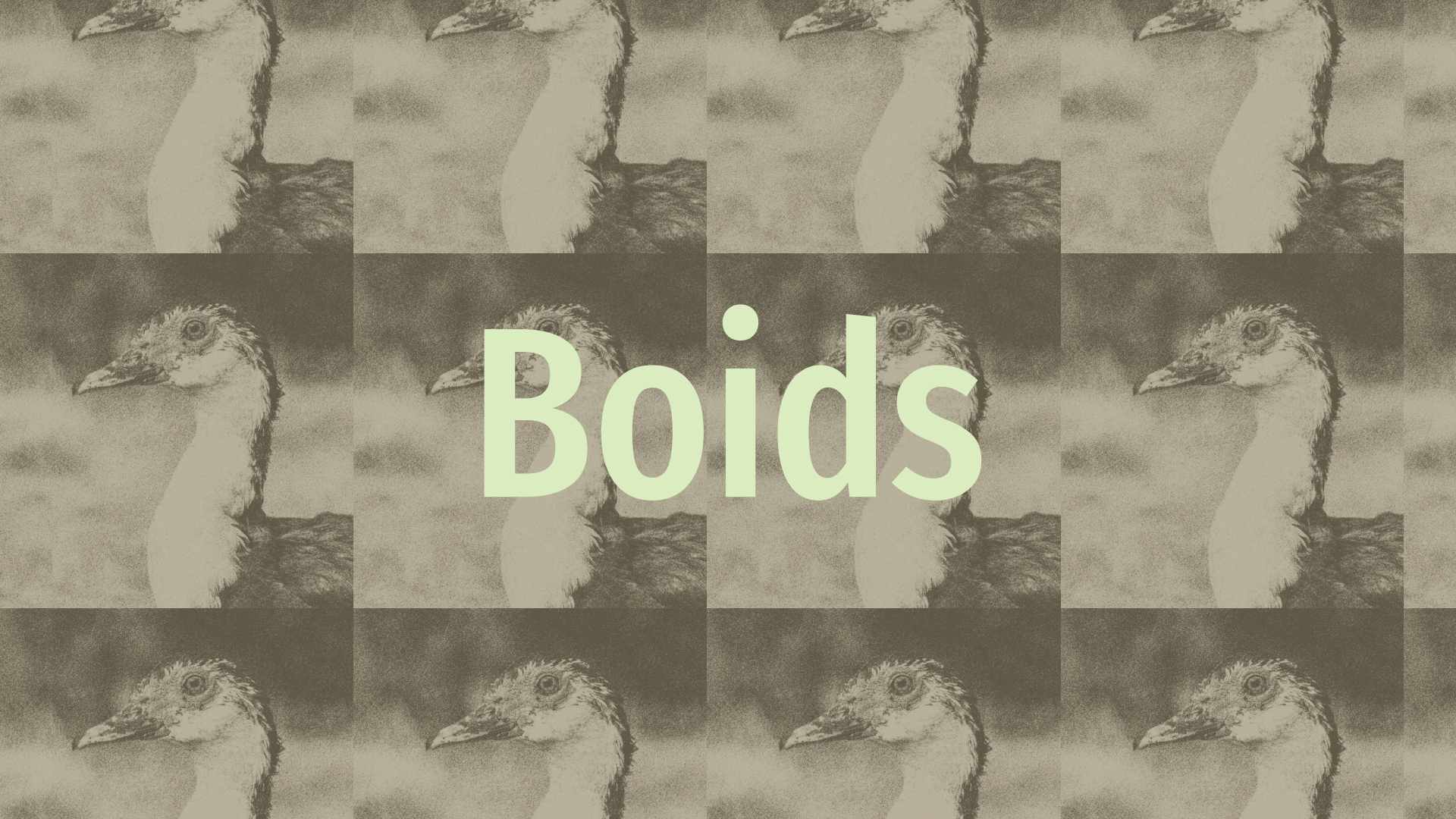


Birds altruism

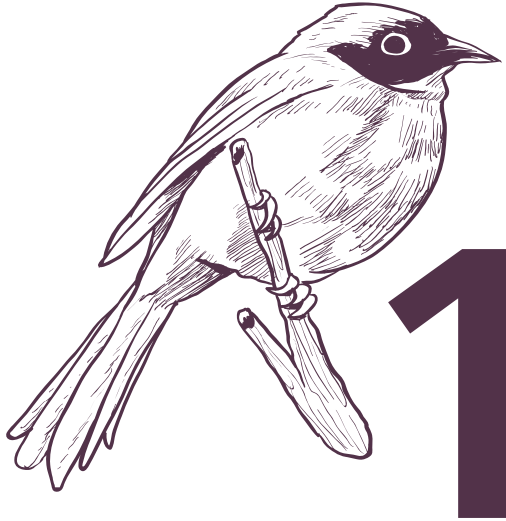
Benefits of one bird are greater
Don't need to fight for food

Key to the effectiveness

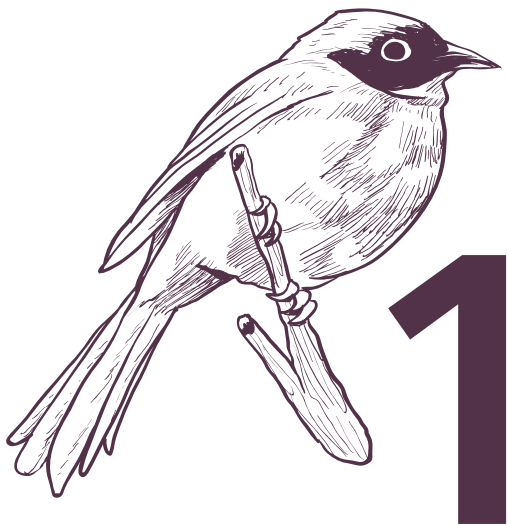
May die without food finds
Play by rules
Sharing information with relatives
Chance to survive increase

The background of the image is a 3x4 grid of 12 identical, slightly desaturated and textured portraits of ostrich heads. Each head is shown in profile, facing left. The word "Boids" is superimposed in the center of the grid, specifically over the four heads in the middle row. The text is a bright, solid yellow-green color, rendered in a clean, sans-serif typeface. The grid pattern is subtle, with the individual portraits slightly offset from each other to create a sense of depth and repetition.

Boids



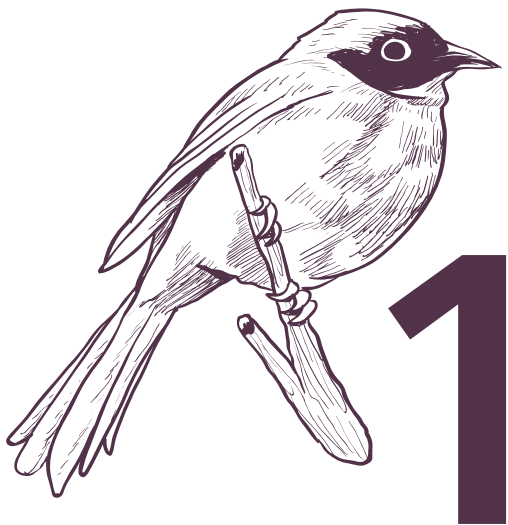
Each bird strives to avoid
bumping into other birds



Each bird strives to avoid
bumping into other birds



Each bird was moving in the
same direction as the nearby
birds



Each bird strives to avoid bumping into other birds



Each bird was moving in the same direction as the nearby birds



The birds tended to move at the same distance from each other

Boids Boids Boid



Boids Boids Boid

The classical particle swarm algorithm

Controlling distances between birds
Maintain the optimal distance



Move towards
optimal solution



Exchange information
with neighbors



Evaluation of a single particle

Function on current position. Remember the best individual solution.

Updating particle velocities and positions

Bases on 3 factors: inertia, own and swarm best solutions.

01

02

03

04

05

Initialization

Set of particles, each of which has random positions and velocities in space. Each particle has its own score.

The Global best solution

Based on all the particles, we choose the globally best solution..

Repetition

The process is repeated: new position, new own and swarm best solution. Stop when reach the stop criterion.



Personal
best location



PSO strategy

They need to communicate

Team
best location

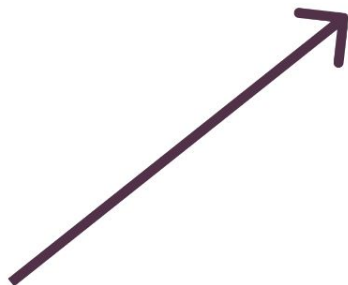


Current
direction

Current
direction



Personal
best location



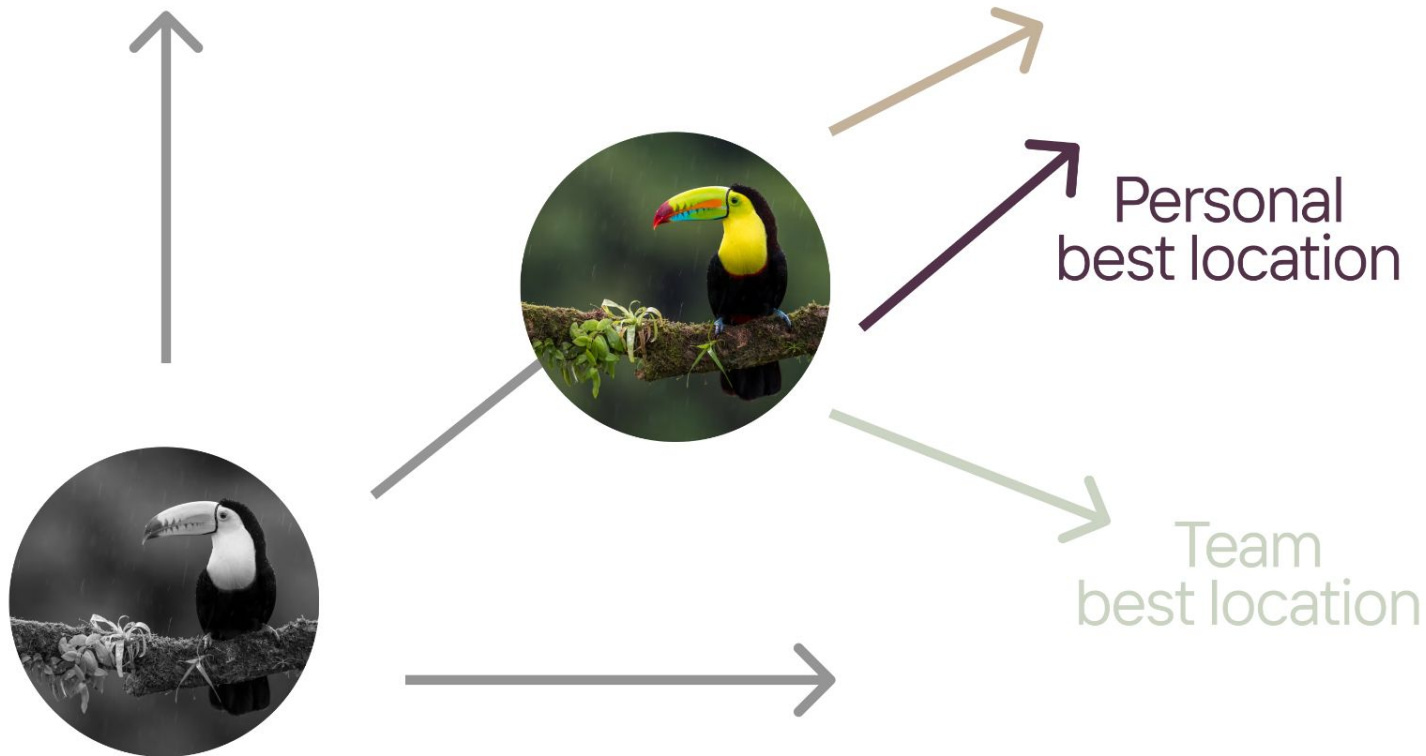
Team
best location



1 day



2 day



Current
direction



Personal
best location

1 day



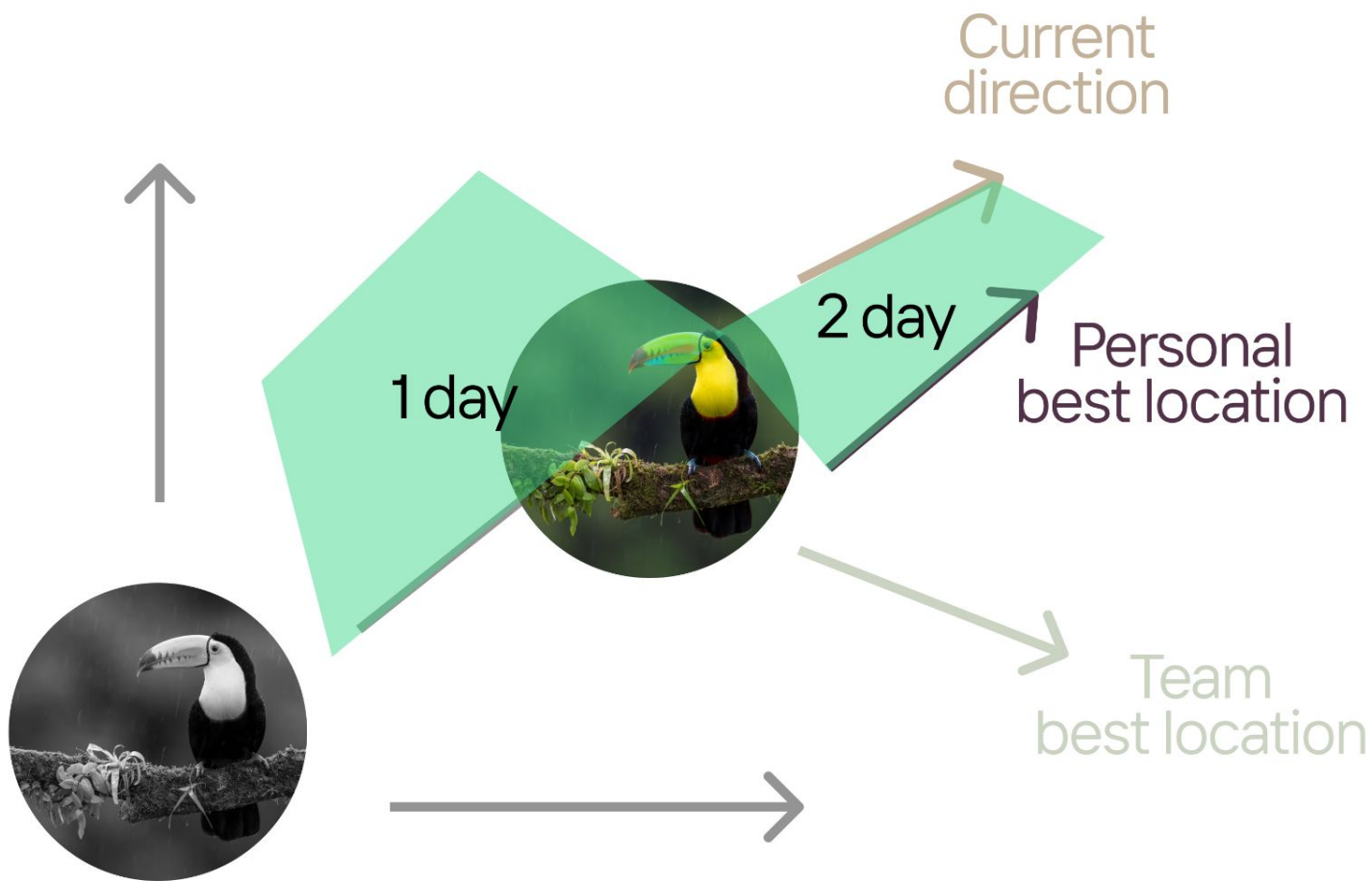
Team
best location



1 day

Every day cover
some territory





2 day

Every day cover
some territory





Every day cover
some territory

Every day make
the chance to
find nest higher



1 day

1 day

5 day

3 day



Next velocity

$$V_{i,t+1} = A_c V_{i,t} + C_p r (p_i - x_{i,t}) + C_g r (g - x_{i,t})$$

Inertia parameter
 Personal best solution
 Weight factor
 Distance to the global best
 Current velocity
 Distance to the personal best
 Weight factor
 Global best solution

Inertia
 Cognitive
 component
 Social
 component

PSO strategy

Position can be find easily

$$\overset{\text{Position in day } d+1}{X_i^{d+1}} = \overset{\text{Position in day } d}{X_i^d} + \overset{\text{Velocity in day } d+1}{V_i^{d+1}}$$

PSO strategy

Position can be find easily

$$X_i^{d+1} = X_i^d + V_i^{d+1}$$



$$X_i^d + V_i^{d+1} \dashrightarrow X_i^{d+1} \dashrightarrow$$

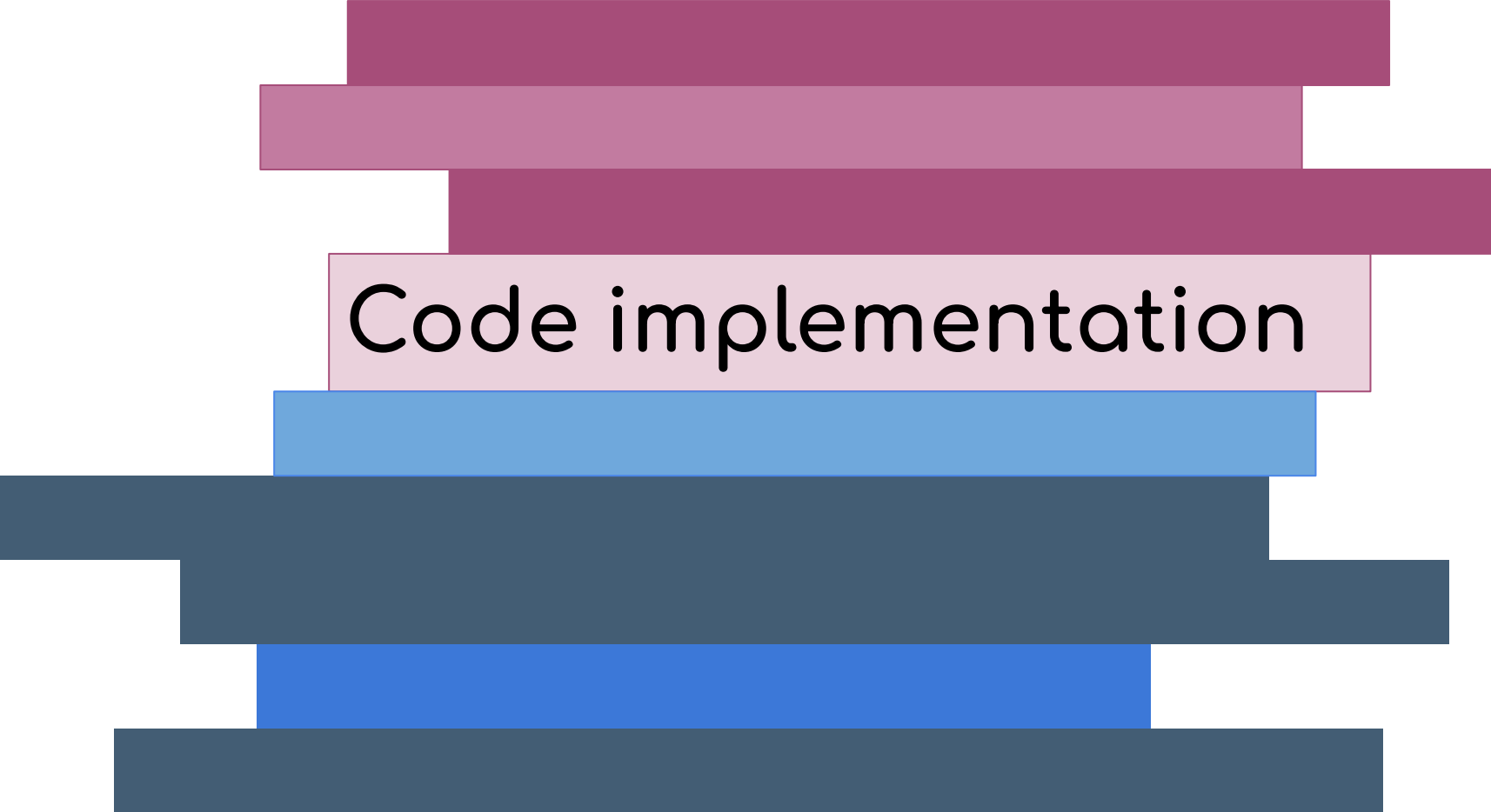


Let's sum up

1 Collective behavior of some agents that can interact with each other to solve a specific task successfully

2 At each step, the particles remember the point where the value was the best, and try to return there

3 The particle swarm algorithm is widely used, among others, in machine learning tasks



Code implementation

```
% Parameters
```

```
numParticles = 50; % Number of particles
maxIter = 50;      % Number of iterations
w = 0.5;           % Inertia weight
c1 = 0.9;          % Cognitive (particle's best) weight
c2 = 0.05;         % Social (global best) weight
```

```
% Search space boundaries
```

```
xMin = -15;
xMax = 15;
yMin = -15;
yMax = 15;
```

```
% Random initialization of particle positions and velocities
```

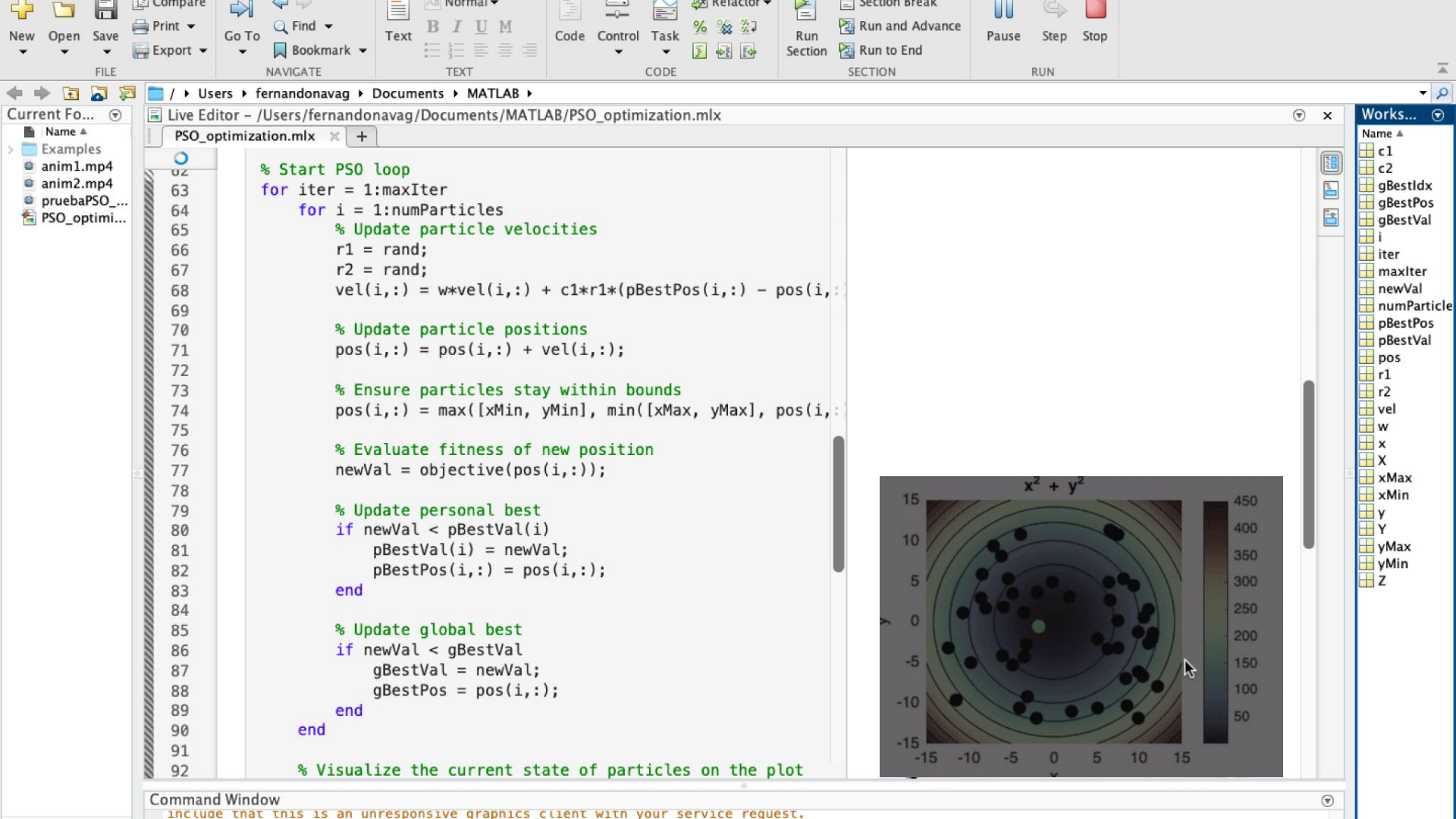
```
pos = [xMin + (xMax-xMin)*rand(numParticles,1), yMin + (yMax-yMin)*rand(numParticles,1)];
vel = zeros(numParticles, 2);
```

```
% Initialize personal bests and global best
```

```
pBestPos = pos; % Personal best positions
pBestVal = arrayfun(@(i) objective(pos(i,:)), 1:numParticles)'; % Evaluate initial positions
[gBestVal, gBestIdx] = min(pBestVal); % Global best value
gBestPos = pos(gBestIdx,:); % Global best position
```

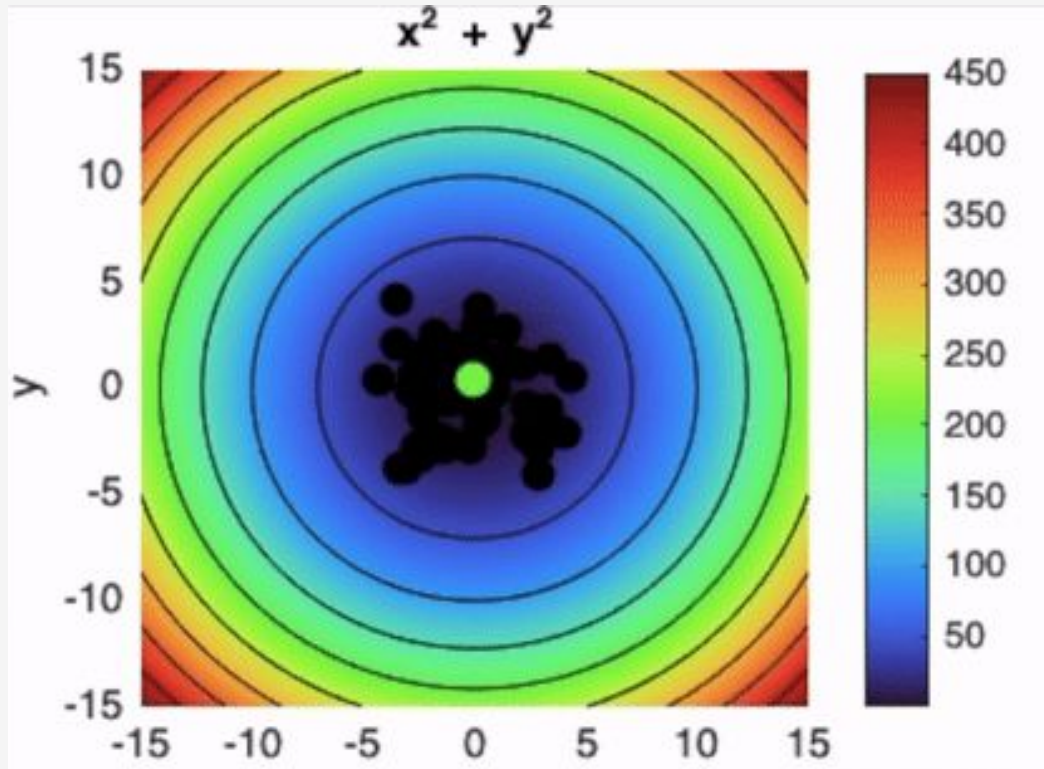
```
% Objective function (the function to minimize)
```

```
function z = objective(p)
    z = p(1)^2 + p(2)^2; % z = x^2 + y^2
    %z = -20 * exp(-0.2 * sqrt(0.5 * ((p(1))^2 + (p(2))^2))) ...
    %      - exp(0.5 * (cos(2 * pi * (p(1))) + cos(2 * pi * (p(2))))) .
    %      + exp(1) + 20;
    %z = ((p(1))^2 + (p(2)) - 11).^2 + ((p(1)) + (p(2)).^2 - 7).^2;
end
```

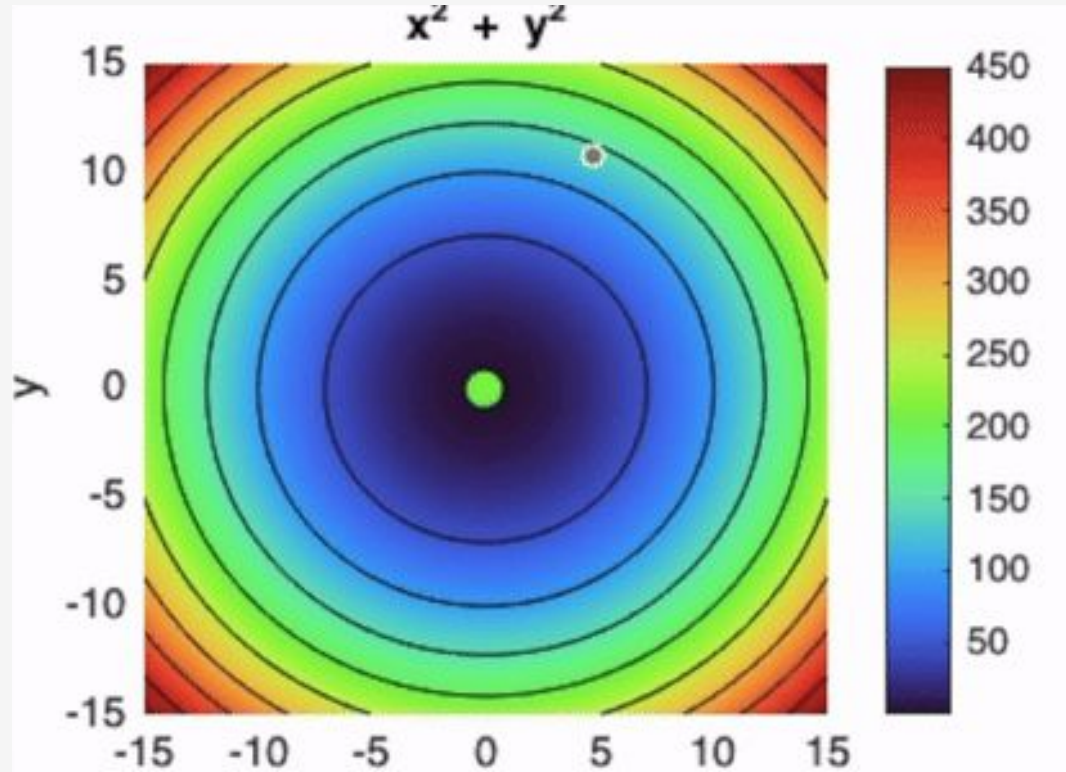
Paraboloid $x^2 + y^2$

numParticles = 50
maxIter = 50
w = 0.8
c1 = 3
c2 = 0.01
Global Best Position:
(-0.0635, 0.3996)
Global Best Fitness: 0.1637

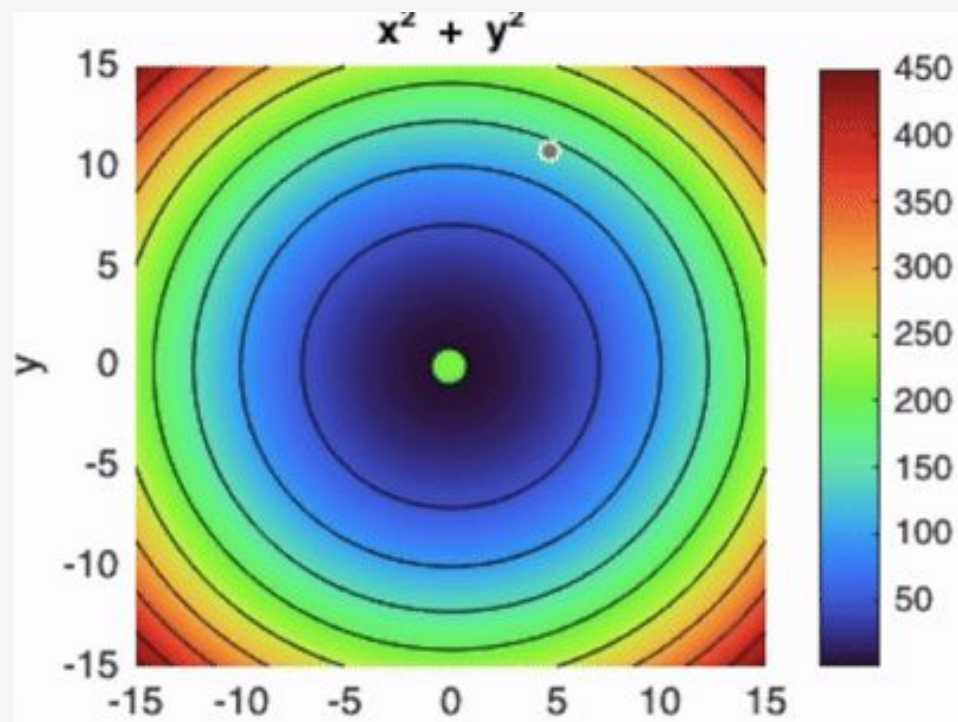
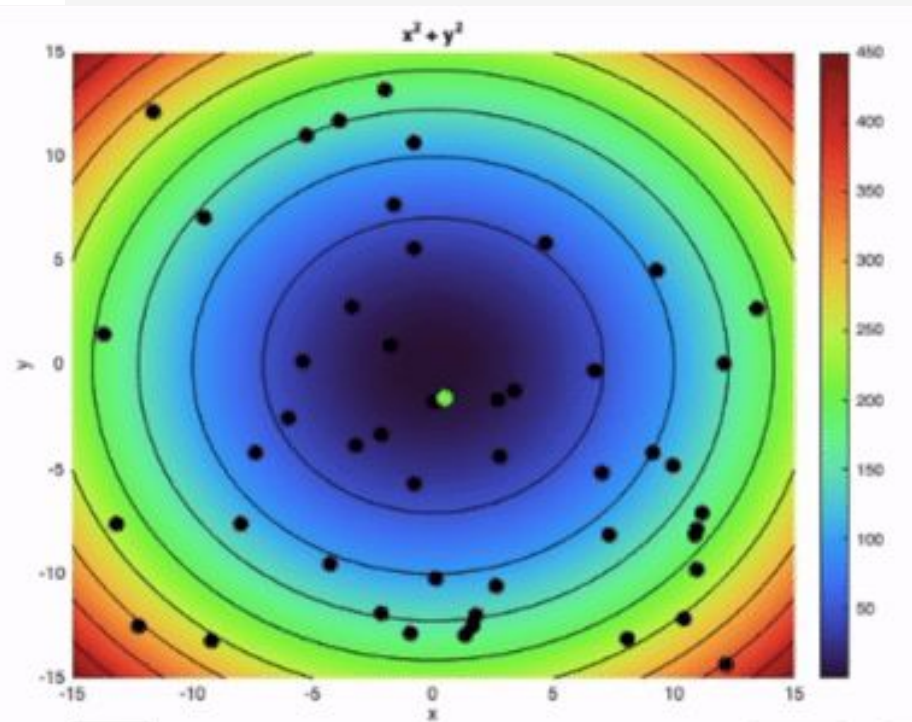


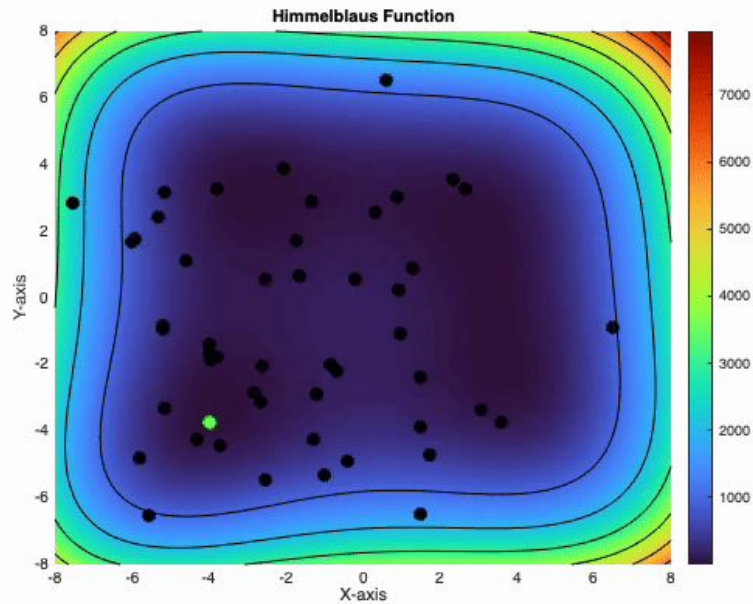
Paraboloid $x^2 + y^2$

numParticles = 50
maxIter = 50
w = 0.6
c1 = 0.6
c2 = 1.2
Global Best Position:
(-0.0000, -0.0000)
Global Best Fitness: 0.0000



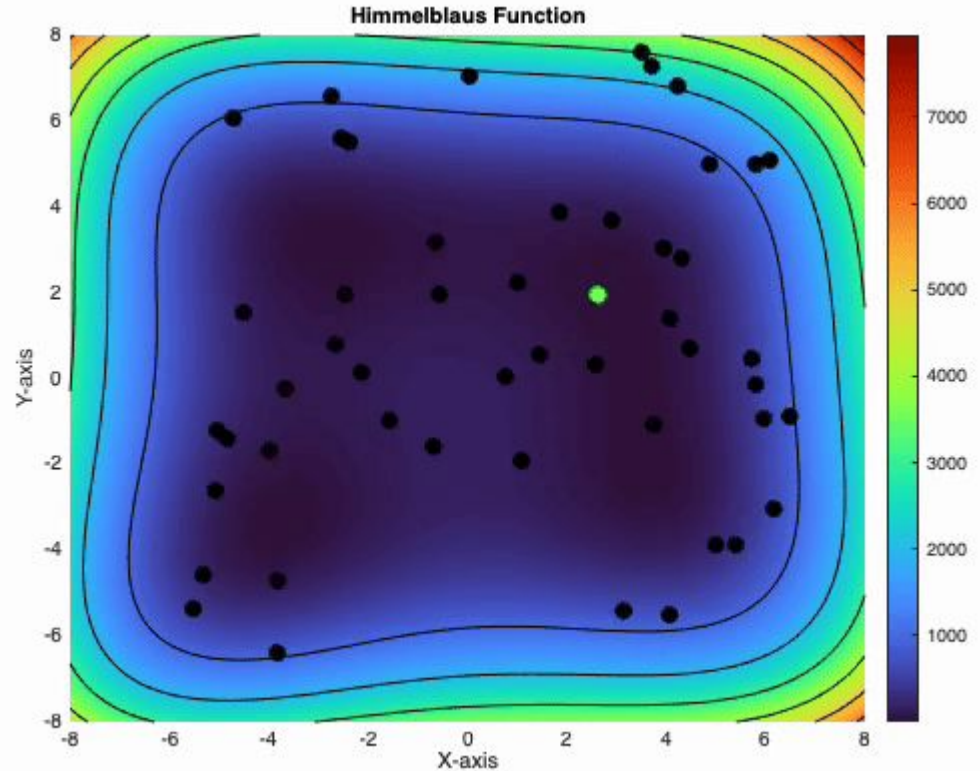
Time comparisons





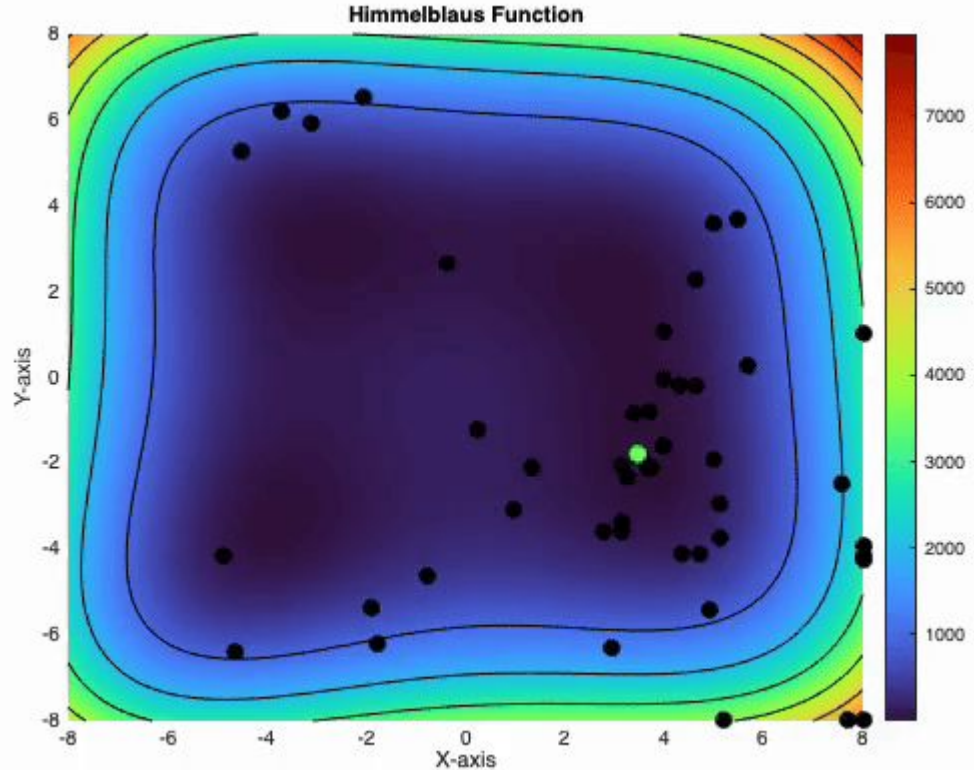
w: 0.5
c1: 2
c2: 0.15
Num of particles: 50
Iterations: 200

Himmelblaus Function



Himmelblaus Function

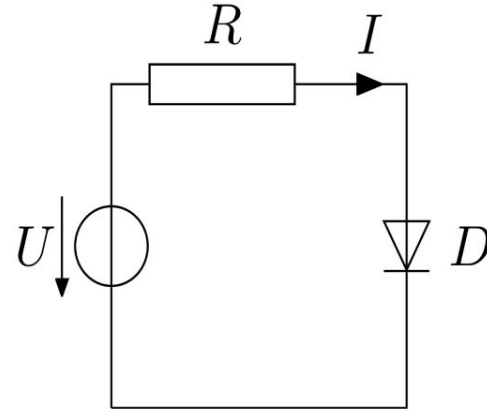
w: 0.3
c1: 0
c2: 2
Num of particles: 50
Iterations: 200



PSO usage

Training a neural network

for optimizing the network's weights and biases



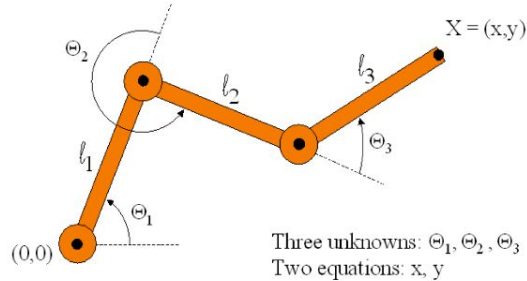
Solving electric circuits

problem consists of analysing a given electric circuit and finding the electric current that flows through it

Inverse Kinematics



- Problem for more complex structures
 - System of equations is usually under-defined
 - Multiple solutions

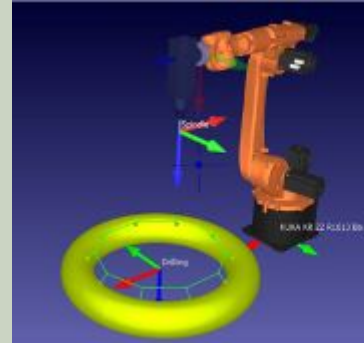


Robotics (inverse kinematics problem)

Inverse Kinematics is one of the most challenging problems in robotics.

The problem involves finding an optimal pose for a manipulator given the position of the end-tip effector as opposed to forward kinematics.

$$c = ||U - vT \log ||IIs|| - RI||$$





Try after class

How does the boids work?